

Branch

分支

一个程序默认是按照代码的顺序执行下来的，有时我们需要选择性的执行某些语句，这时候就需要分支的功能来实现。选择合适的分支语句可以提高程序的效率。

if 语句

基本 if 语句

以下是基本 if 语句的结构。

```
1 if (条件) {  
2     主体;  
3 }
```

if 语句通过对条件进行求值，若结果为真（非 0），执行语句，否则不执行。

如果主体中只有单个语句的话，花括号可以省略。

if...else 语句

```
1 if (条件) {  
2     主体1;  
3 } else {  
4     主体2;  
5 }
```

if...else 语句和 if 语句类似，else 不需要再写条件。当 if 语句的条件满足时会执行 if 里的语句，if 语句的条件不满足时会执行 else 里的语句。同样，当主体只有一条语句时，可以省略花括号。

else if 语句

```
1 if (条件1) {  
2     主体1;  
3 } else if (条件2) {  
4     主体2;  
5 } else if (条件3) {  
6     主体3;  
7 } else {  
8     主体4;  
9 }
```

else if 语句是 if 和 else 的组合，对多个条件进行判断并选择不同的语句分支。在最后一条的 else 语句不需要再写条件。例如，若条件 1 为真，执行主体 1，条件 3 为真而条件 1 和条件 2 都为假，执行主体 3，所有的条件都为假才执行主体 4。

实际上，这一个语句相当于第一个 if 的 else 分句只有一个 if 语句，就将花括号省略之后放在一起了。如果条件相互之间是并列关系，这样写可以让代码的逻辑更清晰。

在逻辑上，大约相当于这一段话：

解一元二次方程的时候，方程的根与判别式的关系：

- 如果 $\Delta < 0$ 方程无解；
- 否则，如果 $\Delta = 0$ 方程有两个相同的实数解；
- 否则 方程有两个不相同的实数解；

switch 语句

```
1 switch (选择句) {
2     case 标签1:
3         主体1;
4     case 标签2:
5         主体2;
6     default:
7         主体3;
8 }
```

switch 语句执行时，先求出选择句的值，然后根据选择句的值选择相应的标签，从标签处开始执行。其中，选择句必须是一个整数类型表达式，而标签都必须是整数类型的常量。例如：

```
1 int i = 1; // 这里的 i 的数据类型是整型，满足整数类型的表达式的要求
2
3 switch (i) {
4     case 1:
5         cout << "OI WIKI" << endl;
6 }
```

```
1 char i = 'A';
2
3 // 这里的 i 的数据类型是字符型，但 char
4 // 也是属于整数的类型，满足整数类型的表达式的要求
5 switch (i) {
6     case 'A':
7         cout << "OI WIKI" << endl;
8 }
```

switch 语句中还要根据需求加入 break 语句进行中断，否则在对应的 case 被选择之后接下来的所有 case 里的语句和 default 里的语句都会被运行。具体例子可看下面的示例。

```
1 char i = 'B';
2
3 switch (i) {
4     case 'A':
5         cout << "OI" << endl;
6         break;
7
8     case 'B':
9         cout << "WIKI" << endl;
10
11     default:
12         cout << "Hello World" << endl;
13 }
```

以上代码运行后输出的结果为 WIKI 和 Hello World，如果不想让下面分支的语句被运行就需要 break 了，具体例子可看下面的示例。

```

1 char i = 'B';
2
3 switch (i) {
4     case 'A':
5         cout << "OI" << endl;
6         break;
7
8     case 'B':
9         cout << "WIKI" << endl;
10        break;
11
12    default:
13        cout << "Hello World" << endl;
14 }

```

以上代码运行后输出的结果为 WIKI，因为 break 的存在，接下来的语句就不会继续被执行了。最后一个语句不需要 break，因为下面没有语句了。

处理入口编号不能重复，但可以颠倒。也就是说，入口编号的顺序不重要。各个 case（包括 default）的出现次序可任意。例如：

```

1 char i = 'B';
2
3 switch (i) {
4     case 'B':
5         cout << "WIKI" << endl;
6         break;
7
8     default:
9         cout << "Hello World" << endl;
10        break;
11
12    case 'A':
13        cout << "OI" << endl;
14 }

```

switch 的 case 分句中也可以选择性的加花括号。不过要注意的是，如果需要在 switch 语句中定义变量，花括号是必须要加的。例如：

```

1 char i = 'B';
2
3 switch (i) {
4     case 'A': {
5         int i = 1, j = 2;
6         cout << "OI" << endl;
7         ans = i + j;
8         break;
9     }
10
11    case 'B': {
12        int qwq = 3;
13        cout << "WIKI" << endl;
14        ans = qwq * qwq;
15        break;
16    }
17
18    default: {
19        cout << "Hello World" << endl;
20    }
21 }

```

本页面最近更新：2023/3/22 15:46:23, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[aofall](#), [billchenchina](#), [CBW2007](#), [CCXXI](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [H-J-Granger](#), [lr1d](#), [ksyx](#), [mgt](#), [NachtgeistW](#), [orzAtalod](#), [ouuan](#), [SukkaW](#), [Tuffy163](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用